

Rechnergestützte Numerik und Simulation

(am Beispiel von Matlab/Simulink)

Numerik 2:

Approximation von Funktionen

2. Approximation von Funktionen

Allgemeiner Hinweis

- Matlab bietet zahlreiche Befehle die das ingenieurmäßige Arbeiten erleichtern.
- Viele dieser Befehle sind dienen der numerischen Behandlung von Standardproblemen.
- Diese Befehle sind zum Teil in den Toolboxen enthalten und zum Teil Bestandteil des Matlab-Standardpakets.

→ Hier kann nur eine kleiner Umfang von Befehlen vorgestellt werde.

Inhalt

2. Approximation von Funktionen

2.1 Interpolation

2.2 Ausgleichsrechnung

2.3 LUT

2.4 KNN/BFN

2.5 Fuzzy-Logik

2. Approximation von Funktionen – 2.1 Interpolation

Interpolation:

Problem 1:

- Gegeben n Stützstellen $x_1, x_2, x_3, \dots, x_n$ mit Werte $y_1, y_2, \dots, y_3$
- Gesucht: Interpolationsfunktion $f(x)$

$$\begin{array}{ll} \text{mit} & y_1 = f(x_1), y_2 = f(x_2), \dots, y_3 = f(x_3) \\ \text{bzw.} & y_1 \approx f(x_1), y_2 \approx f(x_2), \dots, y_3 \approx f(x_3) \end{array}$$

→ z. B. Interpolation von Messdaten

Problem 2:

- Gegeben: $\tilde{f}(x)$ → aufwendig zu berechnen
- Gesucht: $f(x) \approx \tilde{f}(x)$ → einfach zu berechnen

→ z. B. Kennlinien in μC /Embedded Systemen

2. Approximation von Funktionen – 2.1 Interpolation

2.1.1 Polynom-Interpolation:

- Für $n + 1$ Stützstellen (x_i, y_i) kann das Interpolationspolynom

$$f(x) = p(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n = \sum_{k=0}^n a_k x^k$$

- bestimmt werden mit

$$\begin{pmatrix} 1 & x_0 & \dots & x_0^n \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_n & \dots & x_n^n \end{pmatrix} \begin{pmatrix} a_0 \\ \vdots \\ a_n \end{pmatrix} = \begin{pmatrix} y_0 \\ \vdots \\ y_n \end{pmatrix}$$

- Alternativ kann die Lagrangesche-Interpolationsformel verwendet werden:

$$p(x) = \sum_{i=0}^n f(x_i) \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

- Effizienter als diese beiden Methoden ist allerdings der Algorithmus nach Newton.

2. Approximation von Funktionen – 2.1 Interpolation

Newton'sche Interpolationsformel:

$$p(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \dots \\ + a_n(x - x_0)(x - x_1) \cdot \dots \cdot (x - x_{n-1})$$

Bestimmung der Koeffizienten durch Einsetzen:

$$x = x_0: \quad p(x_0) = a_0 = y_0$$

$$x = x_1: \quad p(x_1) = a_0 + a_1(x_1 - x_0) \quad \Leftrightarrow \quad a_1 = \frac{y_1 - y_0}{x_1 - x_0}$$

$$x = x_2: \quad p(x_2) = a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) \\ \Leftrightarrow \quad a_2 = \frac{1}{x_2 - x_1} \left(\frac{y_2 - y_0}{x_2 - x_0} - \frac{y_1 - y_0}{x_1 - x_0} \right)$$

⋮

Allgemein: Schema der "dividierten Differenzen":

$$\begin{array}{l}
 y_0 = a_0 \\
 y_1 \\
 y_2
 \end{array}
 \begin{array}{l}
 \diagup \\
 \diagdown \\
 \diagup \\
 \diagdown
 \end{array}
 \begin{array}{l}
 f[x_0, x_1] = \frac{y_1 - y_0}{x_1 - x_0} = a_1 \\
 f[x_1, x_2] = \frac{y_2 - y_1}{x_2 - x_1}
 \end{array}
 \begin{array}{l}
 \diagup \\
 \diagdown
 \end{array}
 f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = a_2$$

2. Approximation von Funktionen – 2.1 Interpolation

Bsp.:

x_i	-1	0	1	2
y_i	5	-2	9	-4

$$\begin{array}{rcl}
 & a_0 & \\
 5 & \searrow & \\
 -2 & \searrow & \frac{-2-5}{0-(-1)} = -7 \quad a_1 \\
 9 & \searrow & 9 \quad a_2 \\
 -4 & \searrow & -12 \quad a_3 \\
 & & -7
 \end{array}$$

$$p(x) = 5 - 7(x+1) + 9(x+1)x - 7(x+1)x(x-1)$$

2. Approximation von Funktionen – 2.1 Interpolation

Satz: Interpolationsfehler

- Sei $f: [a, b] \rightarrow \mathbb{R}$ und $(x_i, y_i = f(x_i))$, $i = 0, \dots, n$ die $n + 1$ Stützstellen und $p(x)$ das Interpolationspolynom vom Grad n , dann gilt

$$\exists \xi \in [a, b] \quad \forall x \in [a, b] \quad \text{mit} \quad f(x) - p(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^n (x - x_i)$$

2. Approximation von Funktionen – 2.1 Interpolation

2.1.2 Polynome und Polynominterpolation in Matlab

MATLAB unterstützt verschiedene Funktionen für das Arbeiten mit Polynomen, wie z.B. Berechnung der Nullstellen, Auswertung und Differentiation. Für fortgeschrittene Anwendungen werden Funktionen wie das „Fitten“ und die Partialbruchzerlegung zur Verfügung gestellt. Es folgt eine Übersicht der vorhandenen Funktionen:

Funktion	Beschreibung
conv	Multiplikation von Polynomen
deconv	Division von Polynomen
poly	Berechnung des charakteristischen Polynoms
polyder	Berechnung der Ableitung eines Polynoms
polyfit	Annähern einer Datenreihe durch ein Polynom
polyval	Auswertung eines Polynoms mit skalaren Werten
polyvalm	Auswertung eines Polynoms $\mathbf{p} = f(\mathbf{x})$ bei dem $\mathbf{x}$ eine quadratische Matrix darstellt
residue	Partialbruchzerlegung
roots	Berechnung der Nullstellen

2. Approximation von Funktionen – 2.1 Interpolation

`polyval`-Befehl:

Ein Polynom kann in MATLAB mithilfe eines Zeilenvektors `p` definiert werden. Die Elemente dieses Vektors stellen dabei die Koeffizienten der Potenzen in sinkender Reihenfolge dar:

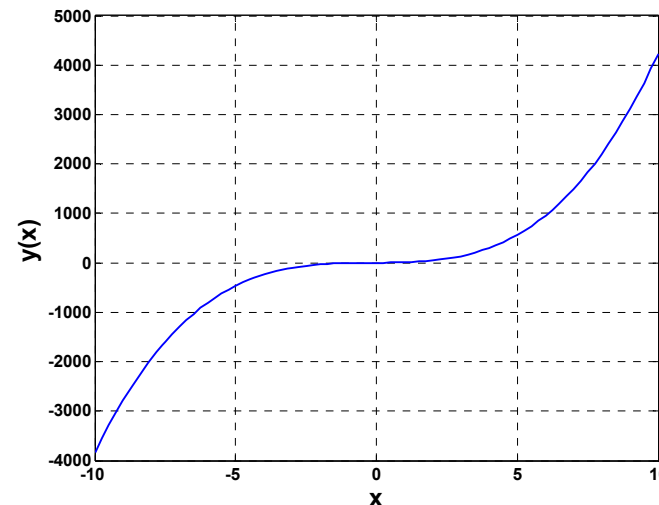
$$p = [2 \quad -3 \quad 1] \rightarrow y = 2x^2 - 3x + 1 \quad (\text{Polynom 2. Ordnung})$$

Der `polyval`-Befehl gibt bei einem gegebenem Polynom $P(x)$ den Funktionswert an einer bestimmten Stelle x zurück.

Sind `p` und `x` also Vektoren mit i respektive j Elementen, werden mit dem Befehl `y=polyval(p,x)` j Funktionswerte des Polynoms der Ordnung $i - 1$ mit den Koeffizienten `p(i)` berechnet.

Beispiel:

```
x=-10:0.25:10;  
p=[4 2 3 1];  
y=polyval(p,x);  
plot(x,y);  
grid on;  
xlabel('x');  
ylabel('y(x)');
```



2. Approximation von Funktionen – 2.1 Interpolation

roots- und poly-Befehle:

Um die Nullstellen eines Polynoms zu berechnen kann der Befehl `roots` verwendet werden.

Dafür wird nur der das Polynom repräsentierende Vektor `p` benötigt: `r=roots(p)`

Beispiel:

```
>> p=[4 2 3 1];
```

```
>> r=roots(p)
```

```
r =
```

```
-0.0712 + 0.8331i
```

```
-0.0712 - 0.8331i
```

```
-0.3576
```

Es ist möglich, mit dem `poly`-Befehl das charakteristische Polynom `c` einer quadratischen Matrix `A` und anschließend mit `roots(c)` die Eigenwerte zu berechnen (direkte Berechnung der Eigenwerte mit `eig(A)`), oder sich mit `poly(r)` die Koeffizienten eines Polynoms mit den Nullstellen `r` ausgeben zu lassen. `poly` und `roots` sind also Umkehrfunktionen voneinander.

poly/roots-Befehl	eig-Befehl	roots=poly⁻¹
<pre>A=[1 2;3 2]; poles=roots(poly(A)) poles = 4 -1</pre>	<pre>A=[1 2;3 2]; poles=eig(A) poles = -1 4</pre>	<pre>p=[1 -3 -4]; r=roots(p); pn=poly(r) pn = 1 -3 -4</pre>

2. Approximation von Funktionen – 2.1 Interpolation

`polyfit`-Befehl:

Liegt nach einem Versuch mit einem reellen System eine Reihe von i Messdaten vor, ist es z.B. für eine spätere Modellbildung hilfreich diese Datenpunkte durch ein Polynom $P(x)$ n -ter Ordnung darzustellen.

`polyfit` gibt, nach Eingabe der Koordinatenwerte der Datenpunkte und Festlegung der Polynomordnung, die Koeffizienten des approximierenden Polynoms zurück.

Beispiel:

```
x = 0:5;           % x-Werte
y = [-4 -6 -6 -4 0 6]; % Funktionswerte
n = 2;             % Ordnung des zu berechnenden Polynoms
p = polyfit(x,y,n)
```

p =

1.0000 -3.0000 -4.0000

Das approximierende Polynom stellt also die Funktion $y = f(x) = x^2 - 3x - 4$ dar.

2. Approximation von Funktionen – 2.1 Interpolation

Regression: (Ausgleichsrechnung)

Lineare Ausgleichsrechnung

Ansatz: $f(x) = a_1 f_1(x) + a_2 f_2(x) + \dots + a_n f_n(x)$

(Bsp.: $p(x) = a_0 + a_1 x + a_2 x^2 + \dots + a_n x^n$)

Stützstellen (x_i, y_i) $i = 0, \dots, n$

$$E(f) = \sum_{i=0}^n (y_i - f(x_i))^2 \rightarrow \min.$$

$$\frac{\partial E}{\partial a_i} = 0 \quad \text{und} \quad E(\underline{a}) = 0$$

2. Approximation von Funktionen – 2.1 Interpolation

$$E(\underline{t}) = \sum_{i=0}^m \left(y_i - \sum_{j=1}^k a_j \cdot f_j(x_i) \right)^2$$

$$= (\underline{y} - \underline{A} \underline{a})^T (\underline{y} - \underline{A} \underline{a})$$

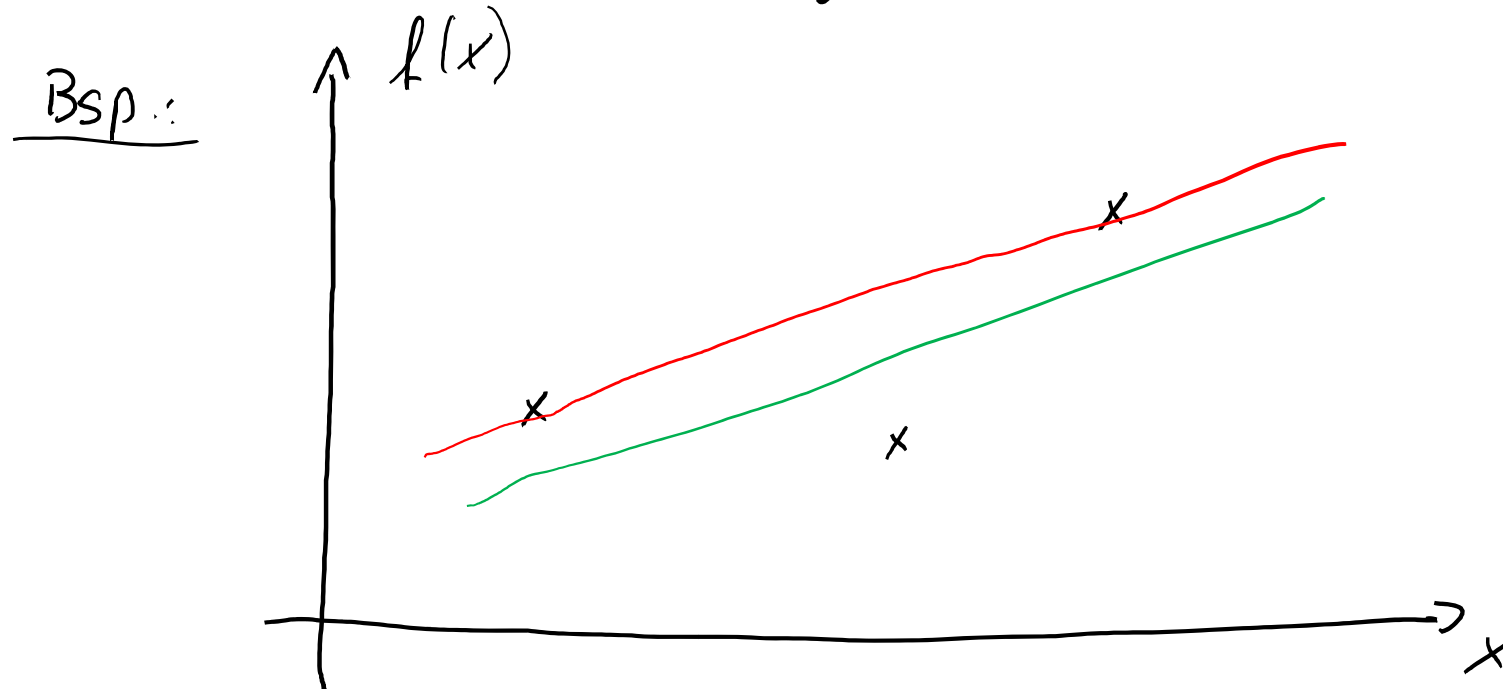
$$\underline{A} = \begin{pmatrix} f_1(x_0) & f_2(x_0) & \dots & f_k(x_0) \\ f_1(x_1) & & & \vdots \\ \vdots & & & \\ f_1(x_m) & \dots & & f_k(x_m) \end{pmatrix}, \quad \underline{a} = \begin{pmatrix} a_1 \\ \vdots \\ a_k \end{pmatrix}, \quad \underline{y} = \begin{pmatrix} y_0 \\ \vdots \\ y_m \end{pmatrix}$$

$$\frac{\partial E}{\partial \underline{a}} = -\underline{A}^T (\underline{y} - \underline{A} \underline{a}) - (\underline{y} - \underline{A} \underline{a})^T \underline{A} = -2 \underline{A}^T (\underline{y} - \underline{A} \underline{a}) = \underline{0}$$

$$\Leftrightarrow \underline{A}^T \underline{y} = \underline{A}^T \underline{A} \underline{a} \rightarrow \text{lin. Gl.-Sys.}$$

2. Approximation von Funktionen – 2.1 Interpolation

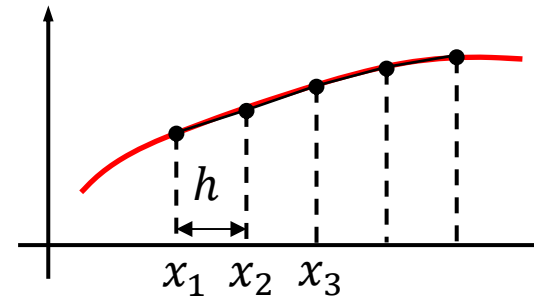
Warum nicht $E(f) = \sum_{i=0}^n |y_i - f(x_i)|$?



2. Approximation von Funktionen – 2.1 Interpolation

2.1.3 Stückweise Interpolation:

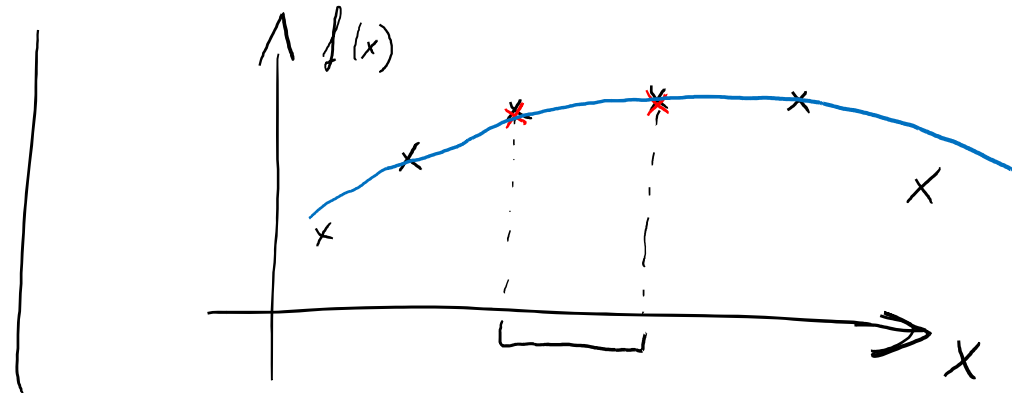
1. Stückweise lineare Interpolation:



$$\text{Fehler: } e = |f(x) - p(x)| \leq \frac{h^2}{8} \max\{|f''(\xi)| \mid \xi \in [a, b]\}$$

2. Stückweise kubische Interpolation → mehr Stützstellen

3. Spline-Interpolation (Tafel)



2. Approximation von Funktionen – 2.1 Interpolation

Spline-Interpolation:

Typisch: kubische Spline-Funktionen $s(x)$

1) $s(x_i) = y_i \quad \forall i = 0 \dots n$

2) s ist Polynom 3. Grades auf jeweils $[x_i, x_{i+1}]$, $i = 0 \dots n-1$

3) s' u. s'' existieren und sind stetig auf $[a, b]$

$n+1$ Stützstellen $\rightarrow n$ kubische Splines $\rightarrow$ $4n$ Koef.

2 Interpolationsstellen $\rightarrow 2n$ Bed.

1. u. 2. Abl. $\rightarrow 2(n-1)$ Bed.

} 2 Freiheitsgrade verbleiben.

2. Approximation von Funktionen – 2.1 Interpolation

2.1.4 Stückweise Interpolation in Matlab

Interpolation ist ein Verfahren zur Schätzung von Werten die zwischen bekannten Datenpunkten liegen. Dabei kann die Verteilung der Datenpunkte beliebige Dimensionen aufweisen. Es folgt eine Übersicht einiger vorhandener Funktionen:

Funktion	Beschreibung
interp1	Eindimensionale Interpolation
interp2	Zweidimensionale Interpolation
interp3	Dreidimensionale Interpolation
interp _n	n-dimensionale Interpolation
spline	Interpoliert mithilfe eines Splines
griddata	Fittet eine Fläche in die Koordinatenpunkte in (x,y,z) und interpoliert die Werte zwischen den Punkten

Die Art der Interpolation kann in jedem Befehl gesondert gewählt werden

2. Approximation von Funktionen – 2.1 Interpolation

`interp1`-Befehle:

Die Funktion `interp1` verwendet Polynome unterschiedlicher Ordnung um zwei Datenpunkte zu verbinden und die Funktionswerte an den angegebene Interpolationspunkten zu berechnen:

```
yi=interp1(x,y,xi,method)
```

<code>yi</code>	– Vektor mit interpolierten Funktionswerten (Ergebnis)
<code>x</code>	– Vektor mit den x-Werten der Datenpunkte
<code>y</code>	– Vektor mit den Funktionswerten der Datenpunkte
<code>xi</code>	– Vektor mit den Interpolationspunkten (Stützstellen)
<code>method</code>	– Art der Interpolation

Liegen die Stützstellen außerhalb des Intervalls der Menge der Datenpunkte, so wird die Funktion automatisch mit der gewählten Methode *extrapoliert*. Die folgenden Interpolationsmethoden sind verfügbar (sortiert nach Rechenaufwand bzw. Genauigkeit):

Interpolationsmethoden:

<code>'nearest'</code>	gibt dem Interpolationswert den Wert des am nächsten liegenden Datenpunktes
<code>'linear'</code>	interpoliert linear zwischen den Datenpunkten
<code>'cubic'</code>	interpoliert die Datenpunkte mit einer quadratischen Funktion
<code>'spline'</code>	interpoliert die Datenpunkte mit einem Polynom mit sich ändernder Ordnung

2. Approximation von Funktionen – 2.1 Interpolation

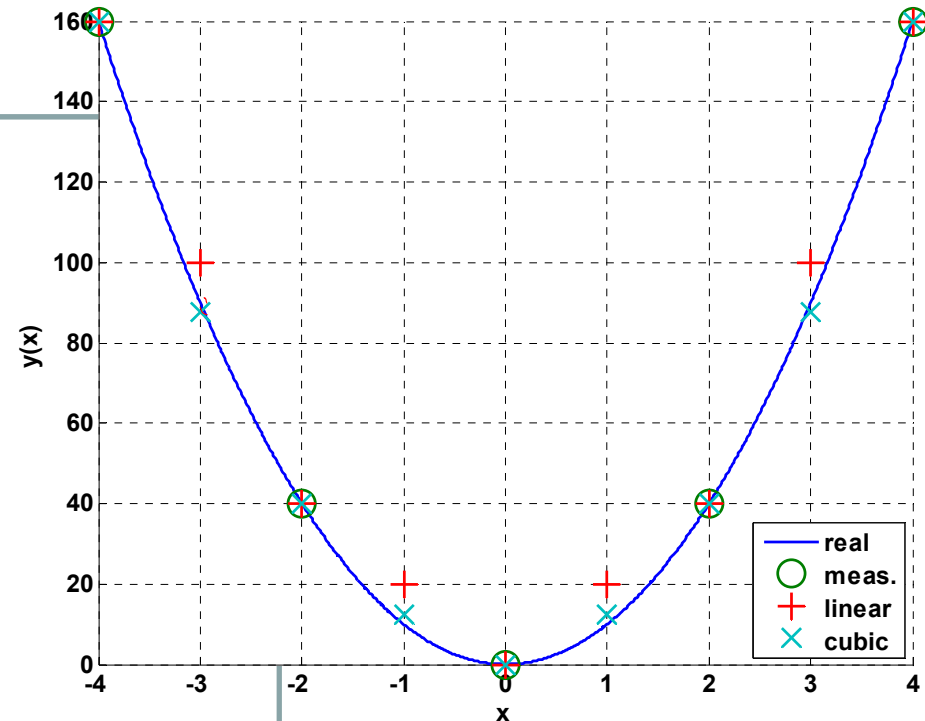
Beispiel: interp1-Befehle

```
% realer Verlauf
xr=-4:0.01:4;
yr=10*xr.^2;

% gemessene Datenpunkte
xm=-4:2:4;
ym=10*xm.^2;

% Interpolation
xi=-4:1:4; % Stützstellen
yil=interp1(xm,ym,xi,'linear');
yic=interp1(xm,ym,xi,'cubic');

% Plot erstellen
plot(xr,yr,xm,ym,'x',xi,yil,'+',xi,yic,'o');
xlabel('x');
ylabel('y(x)');
legend('real','meas.','linear','cubic',4);
grid on;
```



2. Approximation von Funktionen – 2.1 Interpolation

`interp2`-Befehle

Bilden die Datenpunkte eine Fläche im Raum $z = f(x, y)$ so können diese mithilfe des `interp2`-Befehls interpoliert werden.

```
zi=interp2(x,y,z,xi,yi,method)
```

<code>zi</code>	- Matrix mit den interpolierten Funktionswerten
<code>x,y</code>	- Funktionsargumente (mit <code>meshgrid</code> -Format)
<code>z</code>	- Matrix mit Funktionswerten
<code>xi,yi</code>	- Interpolationspunkte (Stützstellen) für interpolierte Funktionswerte (<code>meshgrid</code> -Format)
<code>method</code>	- Interpolationsmethode

Dabei ist darauf zu achten, dass die Funktionsargumente und die Interpolationspunkte monoton sind.

Alternative Schreibweisen:

<code>zi=interp2(z,xi,yi)</code>	nimmt an, dass $x=1:n$ und $y=1:m$, wobei $[m\ n]=\text{size}(z)$ ist
<code>zi=interp2(z,n)</code>	erweitert z mit n Zwischenpunkten

Auch bei diesen beiden Varianten kann die Interpolationsmethode als letztes Funktionsargument in `interp2` festgelegt werden.

2. Approximation von Funktionen – 2.1 Interpolation

Beispiel: Vergleich der Interpolationsmethoden bei `interp2`:

```
% data
[x,y] = meshgrid(-3:1:3);
z = peaks(x,y);           %Beispielfunktion

%interpolation points
[xi,yi] = meshgrid(-3:0.25:3);

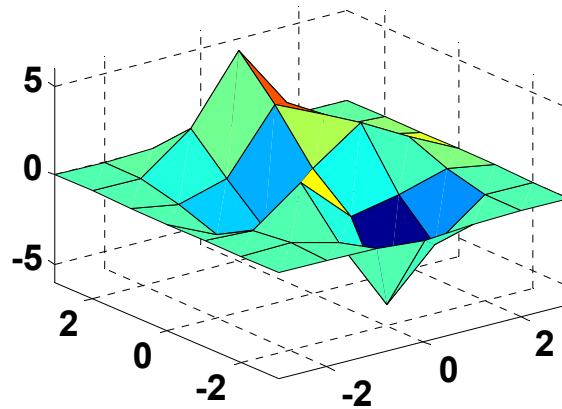
%interpolations
zi1 = interp2(x,y,z,xi,yi,'nearest');
zi2 = interp2(x,y,z,xi,yi,'linear');
zi3 = interp2(x,y,z,xi,yi,'spline');

% plots
subplot(2,2,1)
surf(x,y,z);              % data points
subplot(2,2,2)
surf(xi,yi,zi1);          % nearest-neighbor interpolation
subplot(2,2,3)
surf(xi,yi,zi2);          % linear interpolation
subplot(2,2,4)
surf(xi,yi,zi3);          % spline interpolation
```

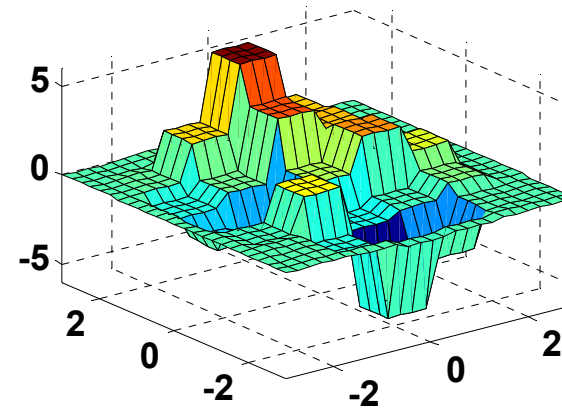
2. Approximation von Funktionen – 2.1 Interpolation

Ergebnis:

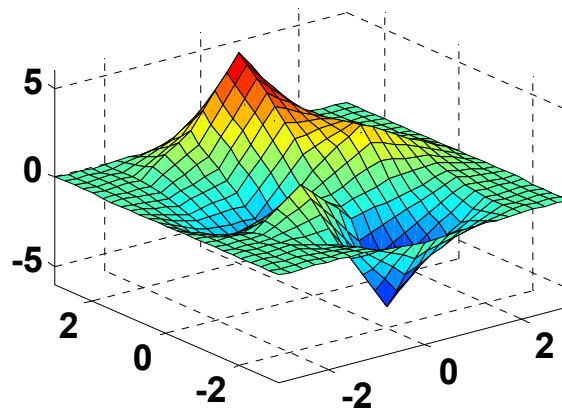
data



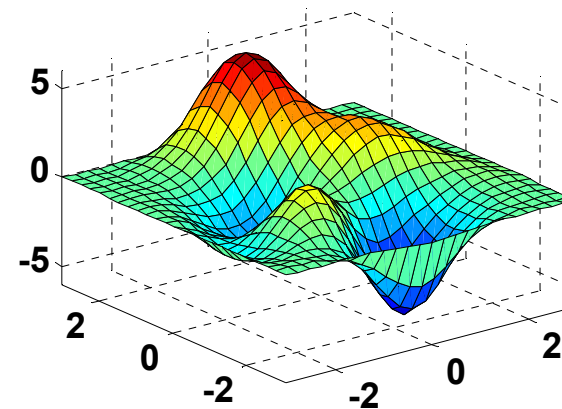
nearest-neighbor



linear



spline



2. Approximation von Funktionen – 2.1 Interpolation

spline-, ppval-Befehle:

Ist es mit dem `polyfit`-Befehl möglich eine Datenreihe durch ein Polynom mit festgelegter Ordnung und Koeffizienten darzustellen, so kann man sie mit dem `spline`-Befehl durch ein Polynom 4. Ordnung mit stückweise unterschiedlichen Koeffizienten annähern. Die Funktion gibt nach Eingabe der x- und y-Werte ein struct zurück.

```
>> x = 0:10;
>> y = cos(x);
>> pp = spline(x,y)
```

```
pp =
    form: 'pp'
 breaks: [0 1 2 3 4 5 6 7 8 9 10]
  coefs: [10x4 double]
pieces: 10
 order: 4
   dim: 1
```

form	– Name des Splines
breaks	– x-Werte der Datenpunkte (Anzahl p)
coefs	– $m \times n$ – Array mit den Koeffizienten von m Polynomen n -ter Ordnung
pieces	– Anzahl der Intervalle zwischen den p Datenpunkten (m)
order	– Ordnung der Interpolationspolynome (n)
dim	– Dimension der Datenreihe

2. Approximation von Funktionen – 2.1 Interpolation

Anschließend können Funktionswerte des Splines mit der Funktion `ppval` (vgl. `polyval`) berechnet werden:

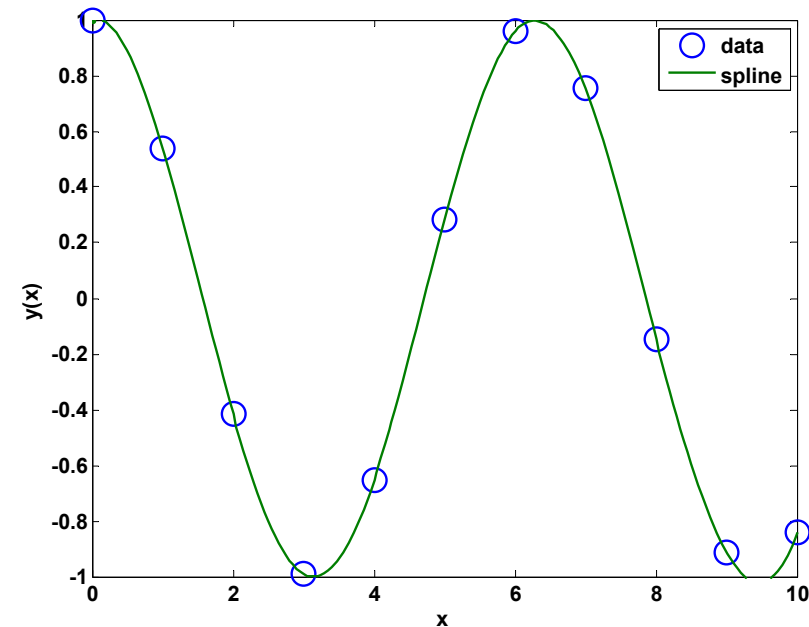
Beispiel:

```
% data creation
x = 0:10;
y = cos(x);

% spline
pp = spline(x,y);

% interpolation
xi=0:0.05:10; % interpolation points
yi=ppval(pp,xi); % interpolant

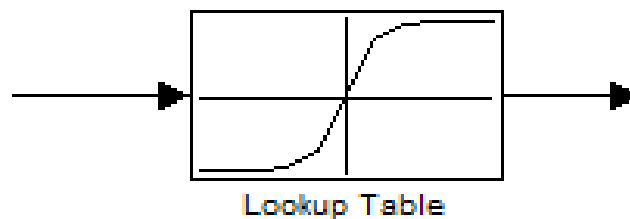
% plot
plot(x,y,'o',xi,yi)
xlabel('x')
ylabel('y(x)')
legend('data','spline')
```



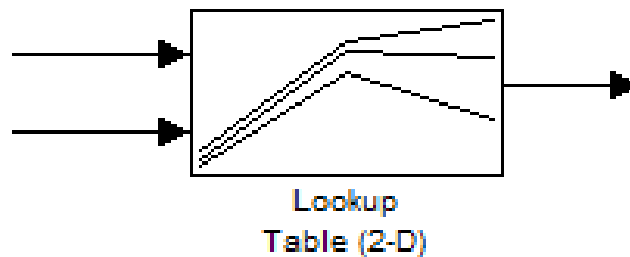
2. Approximation von Funktionen – 2.3 Look-up-Table

Look-up-Table (LUT):

- Tabellen über Stützstellen zur Darstellung nichtlinearer Funktionen
- Anwendung z. B. in Simulink oder in Embedded-Code
- Nutzung der üblichen Interpolations-/Extrapolationsmethoden (s. o.)



*Beispiel –
LUT in Simulink*



2. Approximation von Funktionen – 2.3 Look-up-Table

LUT mit Knicken:

- Soll ein nicht differenzierbare oder nicht stetige Funktionen dargestellt werden (→ LUT mit Knick oder Versprung) empfiehlt sich oft eine Fallunterscheidung.

Problem:

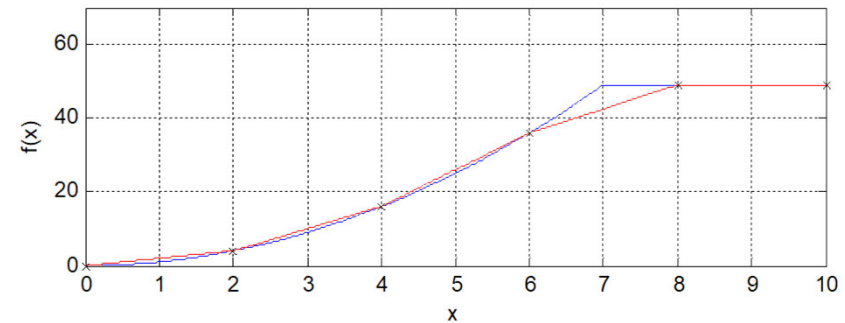
$$f(x) = \begin{cases} x^2 & \text{für } x \leq 7 \\ 49 & \text{für } x > 7 \end{cases}$$

LUT mit Stützstellen 0, 2, 4, 6, 8, 10

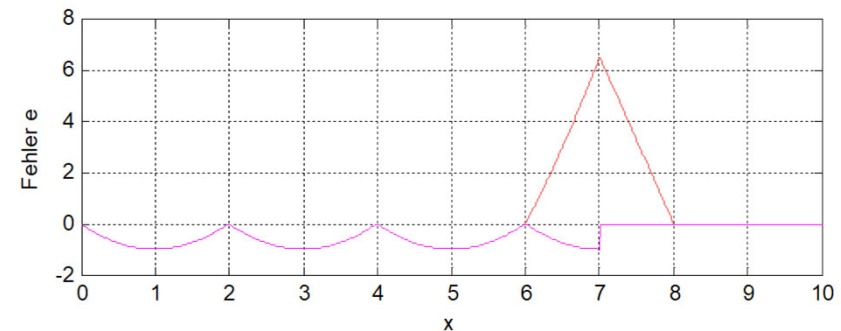
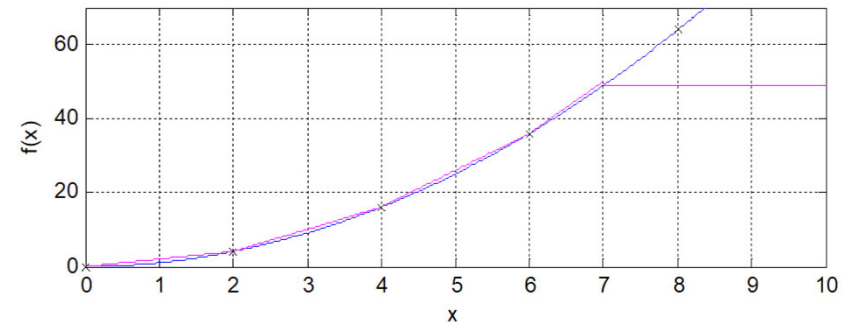
Interpolation mit Fallunterscheidung:

$$f_{LUT} = \begin{cases} \text{interp1}(< x^2 >) & \text{für } x \leq 7 \\ 49 & \text{für } x > 7 \end{cases}$$

Direkte Interpolation der Stützstellen (linear):



Interpolation mit Fallunterscheidung:



2. Approximation von Funktionen – 2.3 Look-up-Table

Beispiel 2:

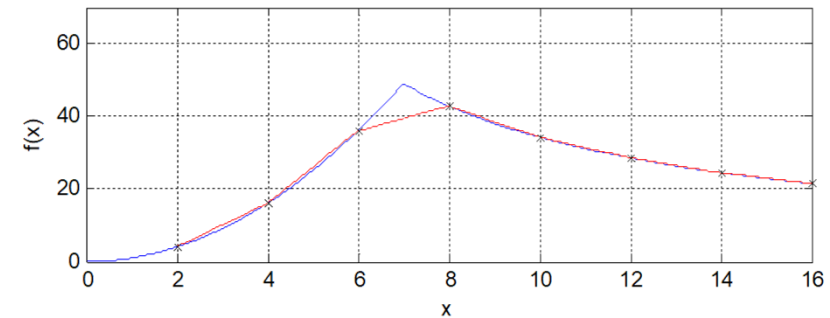
$$f(x) = \begin{cases} x^2 & \text{für } x \leq 7 \\ 7^3/x & \text{für } x > 7 \end{cases}$$

LUT mit Stützstellen 0, 2, 4, 6, 8, 10

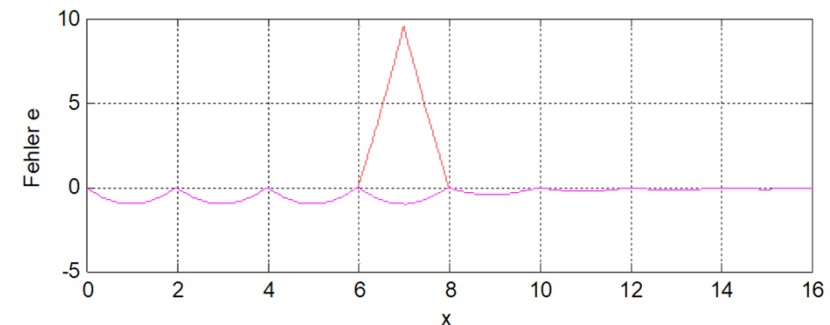
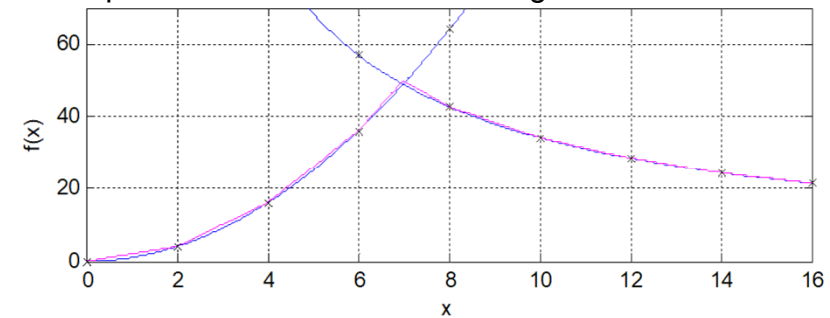
Interpolation mit Fallunterscheidung:

$$f_{LUT} = \begin{cases} \text{interp1}(< x^2 >) & \text{für } x \leq 7 \\ \text{interp1}(< 7^3/x >) & \text{für } x > 7 \end{cases}$$

Direkte Interpolation der Stützstellen (linear):



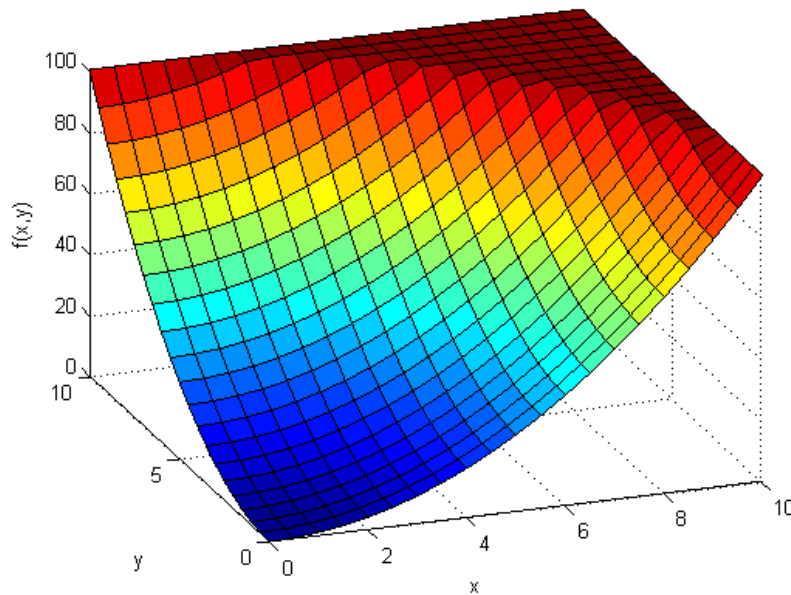
Interpolation mit Fallunterscheidung:



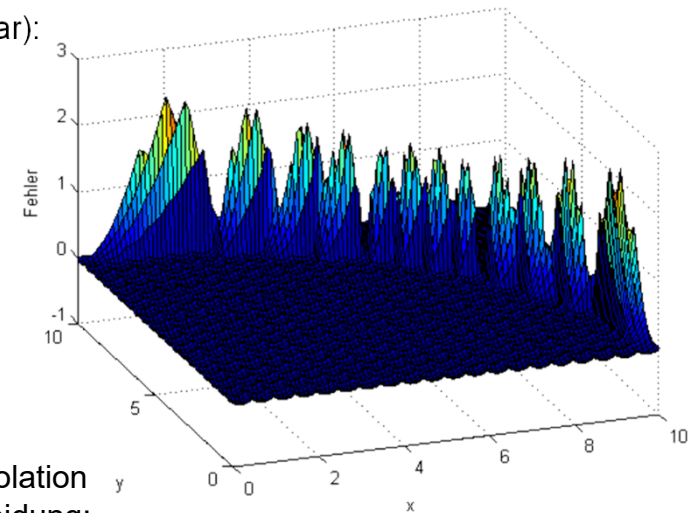
2. Approximation von Funktionen – 2.3 Look-up-Table

Beispiel: 2-dimensionale Funktion:

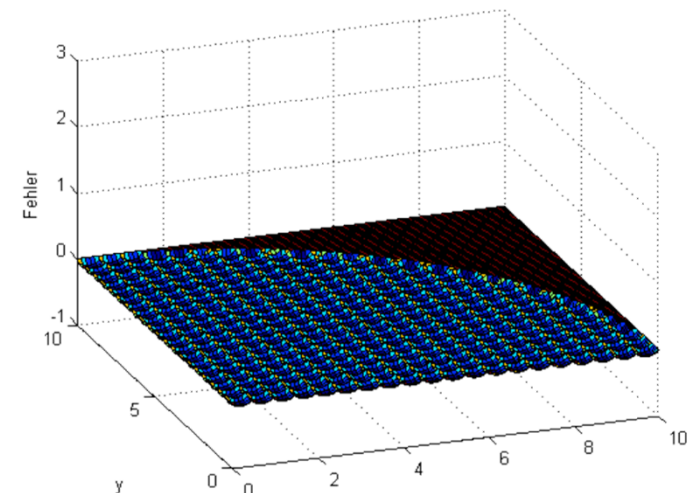
$$f(x, y) = \begin{cases} x^2 + y^2 & \text{für } x^2 + y^2 \leq 100 \\ 100 & \text{für } x^2 + y^2 > 100 \end{cases}$$



Fehler bei direkte Interpolation (linear):



Fehler bei Interpolation mit Fallunterscheidung:



2. Approximation von Funktionen – 2.4 KNN/BFN

Künstliche Neuronale Netze (KNN):

- Oft müssen sehr aufwendige Funktionen abgebildet werden die nur als Messdaten oder Testmuster mit ggf. geringer Datendichte vorliegen oder deren analytische Beschreibung sehr aufwendig ist oder die nur als abstraktes Expertenwissen vorliegen.
- Als Funktionen, also eindeutige Abbildungen einer ggf. auch vektoriellen Größe auf eine andere können in deinem Zusammenhang unterschiedlichste Anwendungen verstanden werden: (Nichtlineare) Regler, Merkmalsbewertung in der Mustererkennung, usw.)

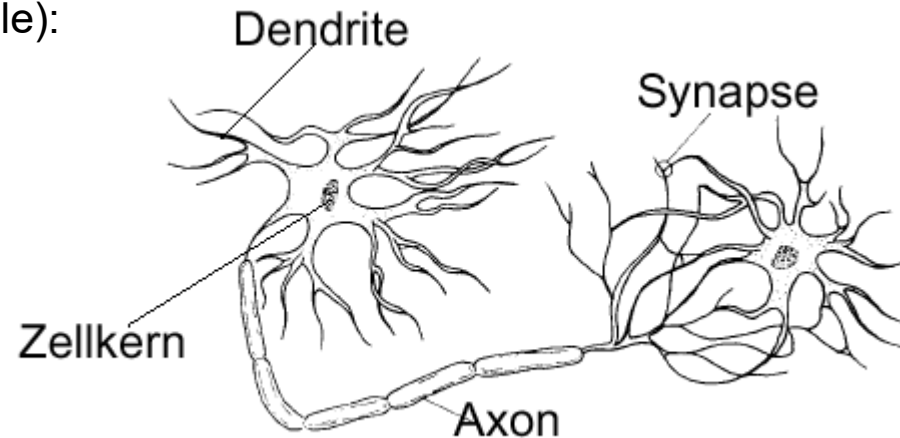
Zwei Ideen:

1. Anstelle einer aufwendigen analytischen Modellbildung soll ein in der Grundstruktur einfaches aber selbstlernendes System verwendet werden. In Anbetracht der erstaunlichen Leistungsfähigkeit biologischer neuronaler Systeme scheint es interessant, deren Funktionsweise nachzubilden → Künstliche neuronale Netze (KNN).
2. Oft sind Funktionen bereits als abstraktes Expertenwissen in den Köpfen der Menschen verfügbar → systematische Extraktion und Umsetzung von Expertenwissen → Fuzzy-Systeme (Kap 4.2.8).

2. Approximation von Funktionen – 2.4 KNN/BFN

Biologische Neuronale Netze

Neuron (Nervenzelle):

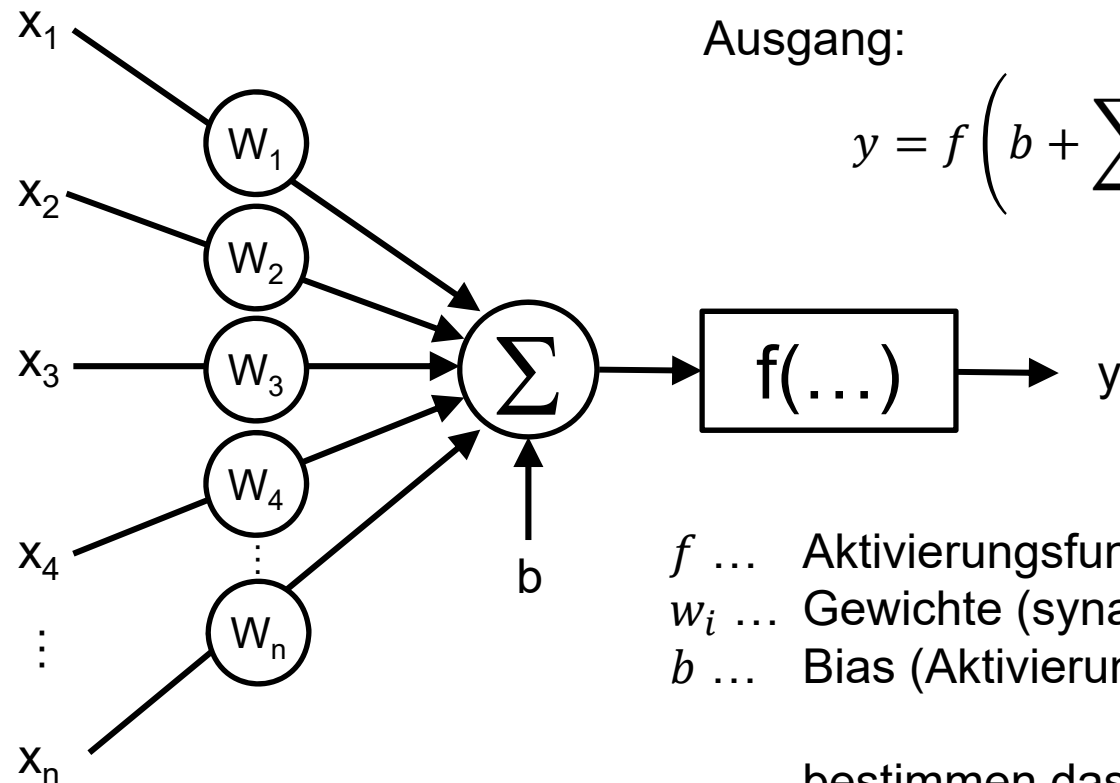


- Ca. 100 Milliarden Neuronen im Gehirn
- Länge des Axon bis ca. 1m
- Dendriten → Empfänger / Synapsen → Sender
- Übertragung durch biochemischen Prozess im synaptischen Spalt zwischen Synapsen und Dendriten (Neurotransmitter, Enzyme)
- Stärke der Nervenreizung ist durch die Zahl der Nerven Pulse pro Zeit (Pulsfrequenzmodulation)

Idee der KNN: Komplexe Zusammenhänge durch „Lernen“ (auch Trainieren) eines der Natur nachempfundenen künstlichen neuronalen Netzes abbilden.

2. Approximation von Funktionen – 2.4 KNN/BFN

Modell eines Neurons:

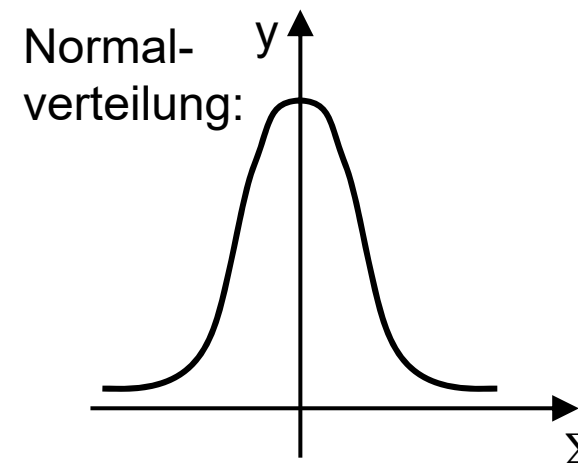
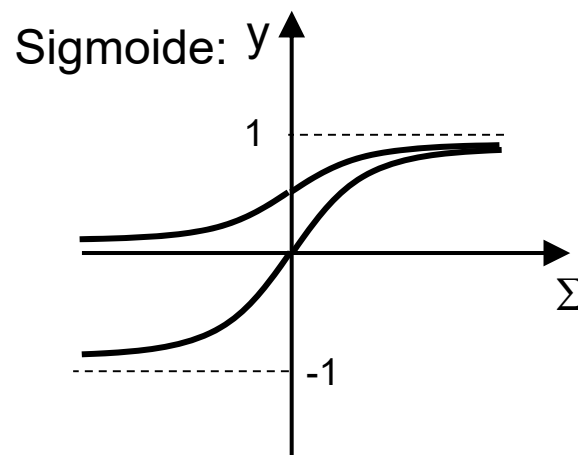
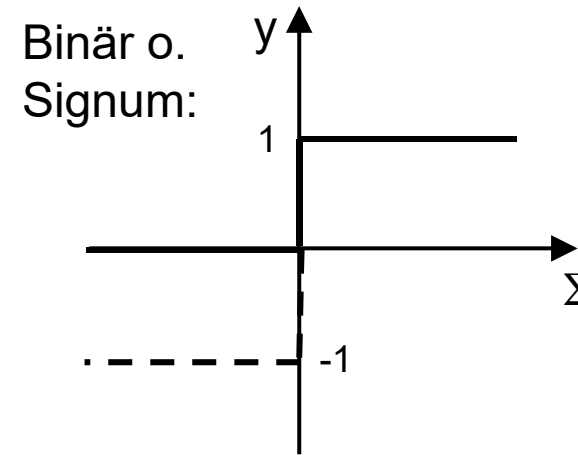
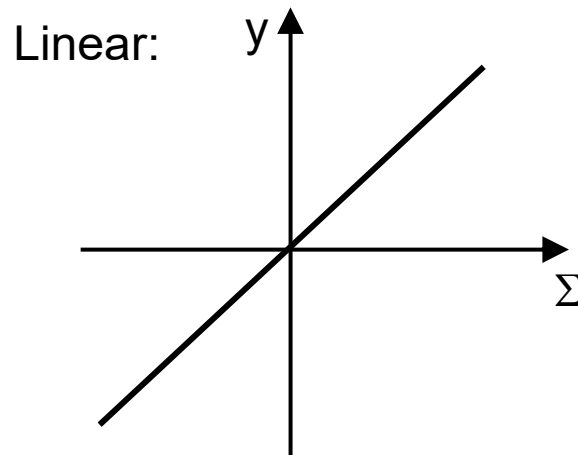


f ... Aktivierungsfunktion
 w_i ... Gewichte (synaptische Gewichte)
 b ... Bias (Aktivierungsbias)

... bestimmen das Verhalten
des Neurons

2. Approximation von Funktionen – 2.4 KNN/BFN

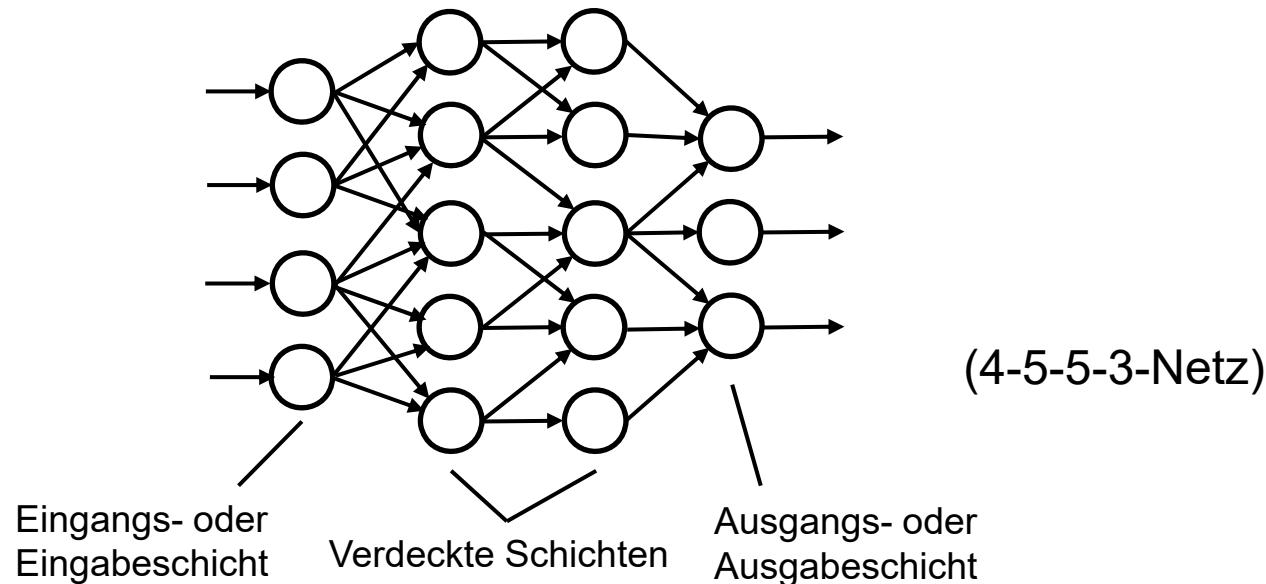
Mögliche Aktivierungsfunktionen (Beispiele):



2. Approximation von Funktionen – 2.4 KNN/BFN

Mögliche Topologie eines KNN (Beispiel):

Azyklisches Netz (vorwärtsgerichtet, kein Rückführung):



Schichtenarchitektur:

- Verbindungen typischerweise nur zwischen aufeinanderfolgenden Schichten
- Verknüpfung muss nicht vollständig sein.
- Mehrere verdeckte Schichten → „Multilayer Perceptron“

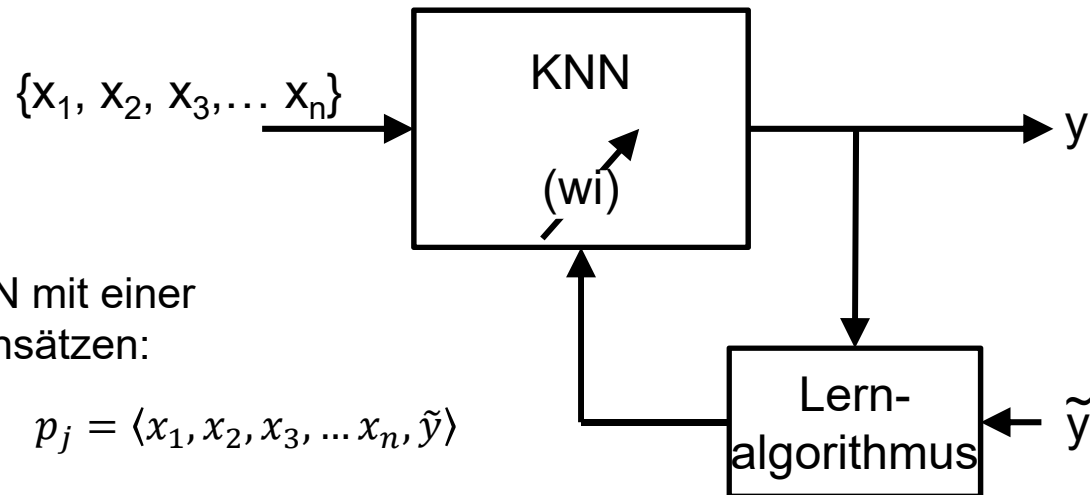
2. Approximation von Funktionen – 2.4 KNN/BFN

Lernen von KNN:

- Trainieren/Lernen des KNN mit einer Menge L von Übungsdatensätzen:

$$L = \{p_1, p_2, p_3, \dots, p_m\} \quad \text{mit} \quad p_j = \langle x_1, x_2, x_3, \dots, x_n, \tilde{y} \rangle$$

- Lernziel: Minimierung des Fehler in $\tilde{y}$ gegenüber y (Bewertung z. B. Summe-Fehlerquadrat)
- Typischer Algorithmus bei mehrschichtigen KNN: Backpropagation-Algorithmus



2. Approximation von Funktionen – 2.4 KNN/BFN

Eigenschaften von KNN:

Positiv:

- Anpassung durch Lernen/Trainieren
- Parallele Bearbeitung
(je nach Implementierung)
- Fehlertoleranz, Robustheit
- Generalisierungsfähigkeit
(Verallgemeinern von Beispielen)

Negativ:

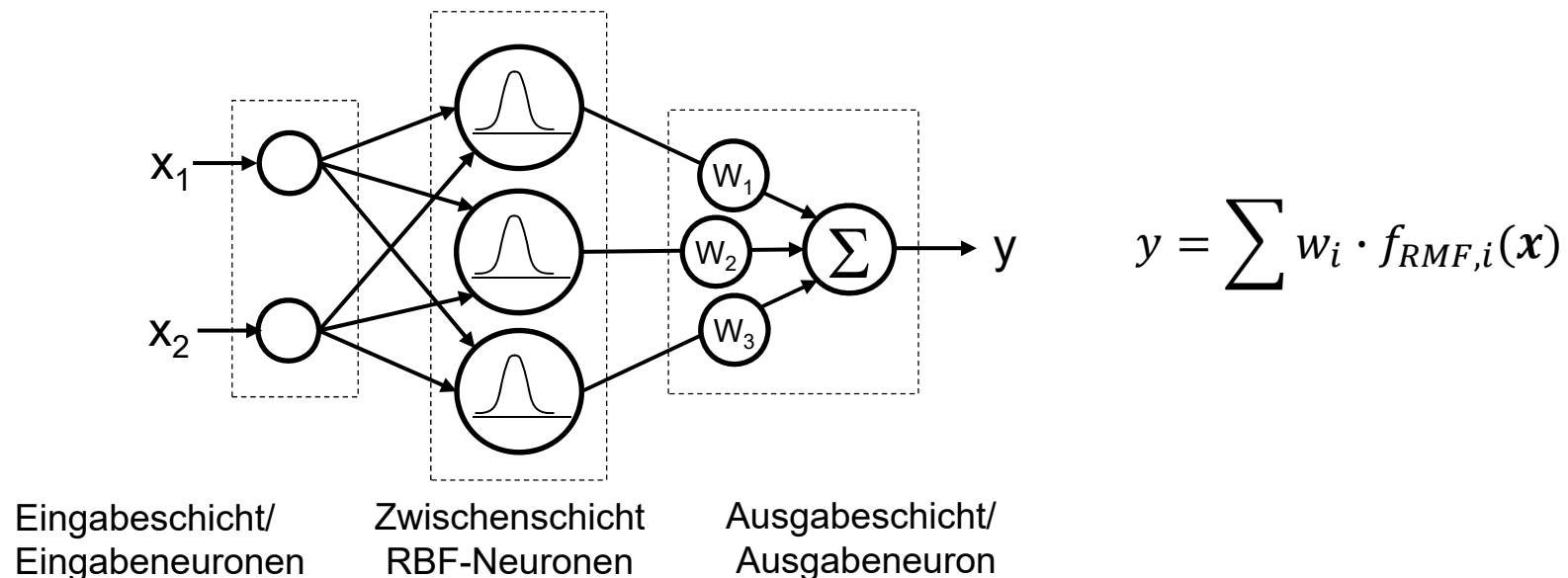
- Lernen erfolgte oft langsam -
eine Vorinitialisierung oft kaum möglich.
- Das gelernte Verhalten hängt von den
Trainingsdaten ab.
- Die inneren Größen sind nicht
interpretierbar (Black-Box).

2. Approximation von Funktionen – 2.4 KNN/BFN

Radiale-Basisfunktionen-Netze (RBF-Netze):

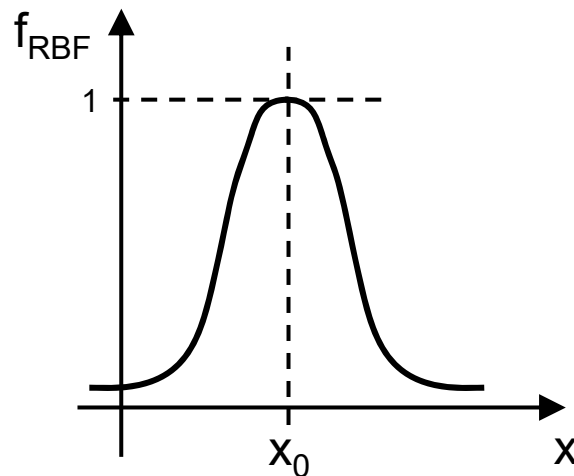
Einfacher KNN-Typ mit einer verdeckten Schicht und radial-symmetrischen Aktivierungsfunktionen f_{RMF} :

- RBF werden lokal trainiert
(im Gegensatz zu den MLP die global trainiert werden)
- internen Werte/Gewichte sind interpretierbar
- Vorinitialisierung u. U. möglich



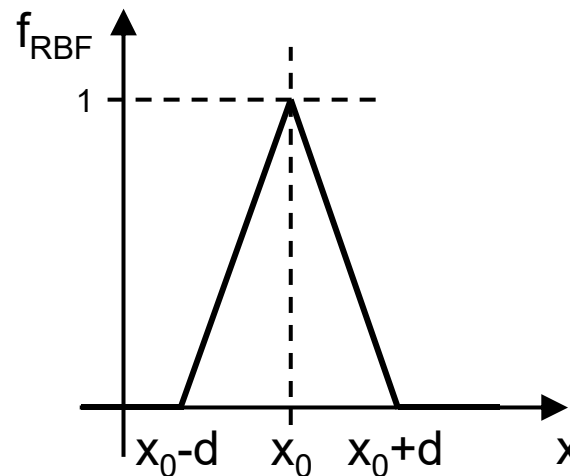
2. Approximation von Funktionen – 2.4 KNN/BFN

Aktivierungsfunktionen von RBF-Netzen (radiale Basisfunktionen):



Gauß-Funktion:

$$f_{RBF,i} = e^{-\frac{(x-x_{0i})^2}{2\sigma_i^2}}$$



Dreiecksfunktion

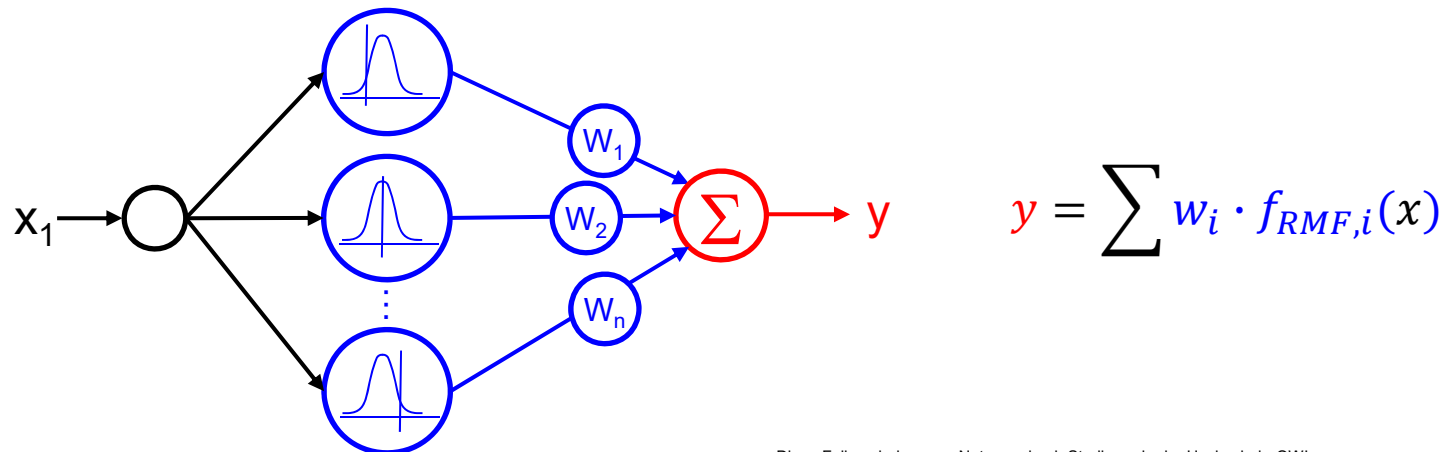
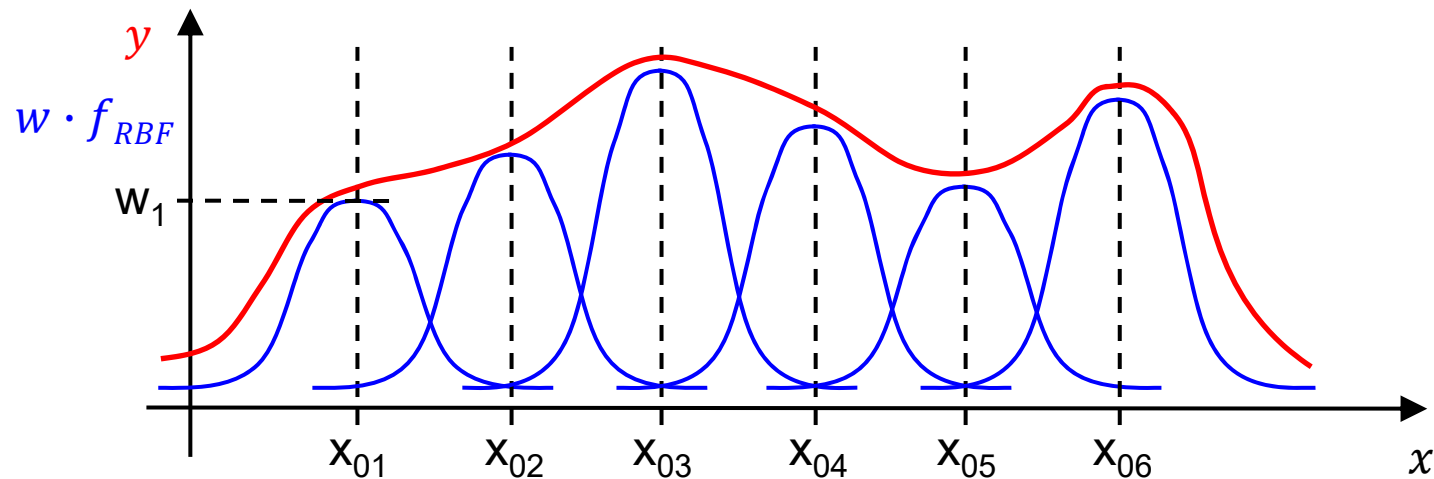
Merkmale:

- Die Basisfunktionen sind symmetrisch um x_0 und haben dort üblicherweise ihr Max. von 1. Die Radien σ bzw. d werden fest gewählt oder beim Lernen adaptiert.
- Bei n Eingängen sind die Basisfunktionen analog als n -dimensionale, radial-symmetrischen Funktionen anzulegen.

2. Approximation von Funktionen – 2.4 KNN/BFN

Darstellung einer Funktion durch ein RBF:

Einfaches Beispiele mit einer Eingangs- und einer Ausgangsgröße:



2. Approximation von Funktionen – 2.4 KNN/BFN

Lernen von RBF-Netzen:

Im allgemeinen werden beim Lernen die Lage der Neuronen (z. B. Clusterbildung) sowie ihre Radien und Gewichte festgelegt.

Sonderfall: Feste Basisfunktionen, z. B. gleichverteilt (Gitter) mit identisch gewählten Radien.

Lernstrategie:

1. Direkte Vorgabe / Initialisierung (nur bei RBF-Netzen möglich):

n Neuronen in der Verdeckten Schicht bei und n Trainingsdatensätze $\rightarrow n$ Gleichungen

$$\begin{aligned} \tilde{y}_1 &= \sum_{i=1}^n w_i \cdot f_{RMF,i}(x_1) \\ \tilde{y}_2 &= \sum_{i=1}^n w_i \cdot f_{RMF,i}(x_2) \\ &\vdots \\ \tilde{y}_n &= \sum_{i=1}^n w_i \cdot f_{RMF,i}(x_n) \end{aligned} \quad \rightarrow \quad \begin{bmatrix} f_{RMF,1}(x_1) & f_{RMF,2}(x_1) & \cdots & f_{RMF,n}(x_1) \\ f_{RMF,1}(x_2) & f_{RMF,2}(x_2) & & \\ \vdots & & \ddots & \vdots \\ f_{RMF,1}(x_n) & & \cdots & f_{RMF,n}(x_n) \end{bmatrix} \cdot \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} = \begin{bmatrix} \tilde{y}_1 \\ \tilde{y}_2 \\ \vdots \\ \tilde{y}_n \end{bmatrix}$$

Lösung des linearen Gleichungssystems liefert die Gewichte w_i .

$\rightarrow$ Analytische Berechnung der Gewichte.

2. Approximation von Funktionen – 2.4 KNN/BFN

2. Adaptives Lernen:

- Trainieren mit Trainingsdatensätzen die in der Lernphase in wahlloser Reihenfolge aufgeschaltet werden.
- Oft als Nachlernen (nach direkter analytischer Berechnung) verwendet.
- Idee ähnlich Newton-Verfahren
(Annahme zunächst: Skalares y und n Trainingsdatensätze bei n Gewichten):

$$\mathbf{e} = \tilde{\mathbf{y}} - \mathbf{y} = \begin{bmatrix} \tilde{y}_1 \\ \vdots \\ \tilde{y}_n \end{bmatrix} - \begin{bmatrix} \sum w_i \cdot f_{RMF,i}(x_1) \\ \vdots \\ \sum w_i \cdot f_{RMF,i}(x_n) \end{bmatrix} = 0 \quad \rightarrow \quad \text{Nullstelle der Fehlerfunktion bzgl. } \mathbf{w}$$

Gedämpftes Newtonverfahren:

$$\mathbf{w}_{n+1} = \mathbf{w}_n - \eta \cdot \mathbf{e}'(\mathbf{w}_n)^{-1} \mathbf{e}(\mathbf{w}_n) = \mathbf{w}_n - \eta \cdot \mathbf{e}'(\mathbf{w}_n)^{-1} \mathbf{e}(\mathbf{w}_n)$$

Abwandlung: Trainingsdatensätze werden nacheinander aufgeschaltet:

$$\mathbf{w}_{k+1} = \mathbf{w}_k + \eta \cdot \begin{bmatrix} f_{RMF,1}(x_n) \\ \vdots \\ f_{RMF,n}(x_n) \end{bmatrix} \cdot (\tilde{y}_n - \sum w_i \cdot f_{RMF,i}(x_n)) \quad \eta = 0 \dots 1 - \text{Lernrate (typ. 0,1)}$$

→ Beliebig viele Trainingsdatensätze können verwendet werden.

2. Approximation von Funktionen – 2.4 KNN/BFN

Anwendung von RBF-Netzen:

- Regelungstechnik:
 - Neuronale Regler → adaptiv, selbstlernend aufgrund des Regelfehlers.
 - Idee: Regelung ohne aufwendige Modellierung bei komplexen, nichtlineare Regelstrecken,
 - In der Praxis: Kompensation von Nichtlinearitäten
 - Überwiegend RBF-Netze
- Mustererkennung:
 - Bewertung extrahierter Merkmale durch KNN
 - Lernen durch Testmuster
 - Überwiegend MLP
- Robotik:
 - Selbstlernende Steuerung bei redundanten Freiheitsgraden
→ Kollisionsvermeidung

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Fuzzy-Logik

Idee: Systematische Extraktion und Umsetzung von Expertenwissen.

Anwendung: Überwiegend Regelungstechnik → „Fuzzy-Regler“

Prinzip: „Abfragen“ des Expertenwissens
über „Linguistischen Variablen“ und „Linguistischen Terme“

Aufbau des Systems (Fuzzy-Reglers):

Eingangsgrößen → „Linguistischer Term“ → Schlussfolgerung → „Lingu. Term“ → Ausgangsgröße

Beispiel:

Raumtemperatur 23°C → „warm“ → Wenn es „warm“ ist dann „kleine“ Heizleistung → „klein“ → 0.2 kW

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Fuzzy-Logik: (Fuzzy $\rightarrow$ „unscharf“)

Klassische Menge (scharfe Menge):

Bei klassischen (scharfen) Mengen gehört ein Element entweder zu einer Menge oder nicht. Die Mengen sind also scharf abgegrenzt.

Problem: Dieser klassische Mengenbegriff ist zwar mathematisch gut fassbar, aber für die Beschreibung „Linguistischer Terme“ eher ungeeignet, da nach allgemeinem Empfinden die „Linguistischer Terme“ nicht scharf abgegrenzt sind.

Bsp.: Ab welchem Wert ist die Raumtemperatur „warm“? Nimmt man 19°C als Grenze, was ist dann mit $18,999^{\circ}\text{C}$?

Fuzzy-Menge (unscharfe Menge):

Der Begriff der Fuzzy-Menge lässt zu, dass ein Element auch nur zum Teil zu einer Menge gehört, um im Sinn der „Linguistischer Terme“ zu beschreiben, dass 19°C schon halbwegs aber noch nicht so richtig „warm“ ist, wie z. B. 22°C . Der Wert 19°C gehört also erst z. B. zu 50% zur „Menge der warmen Temperaturen“. Die Mengen sind damit nicht scharf abgegrenzt $\rightarrow$ „unscharf“.

Die „Unschärfe“ gilt auch für die aus der Mengenzugehörigkeit resultierenden Aussagelogik. Die Aussage „ 19°C ist warm“ oder anders „ 19°C gehört zur „Menge der warmen Temperaturen“ ist also nicht wahr oder falsch sondern z. B. zu 50% wahr.

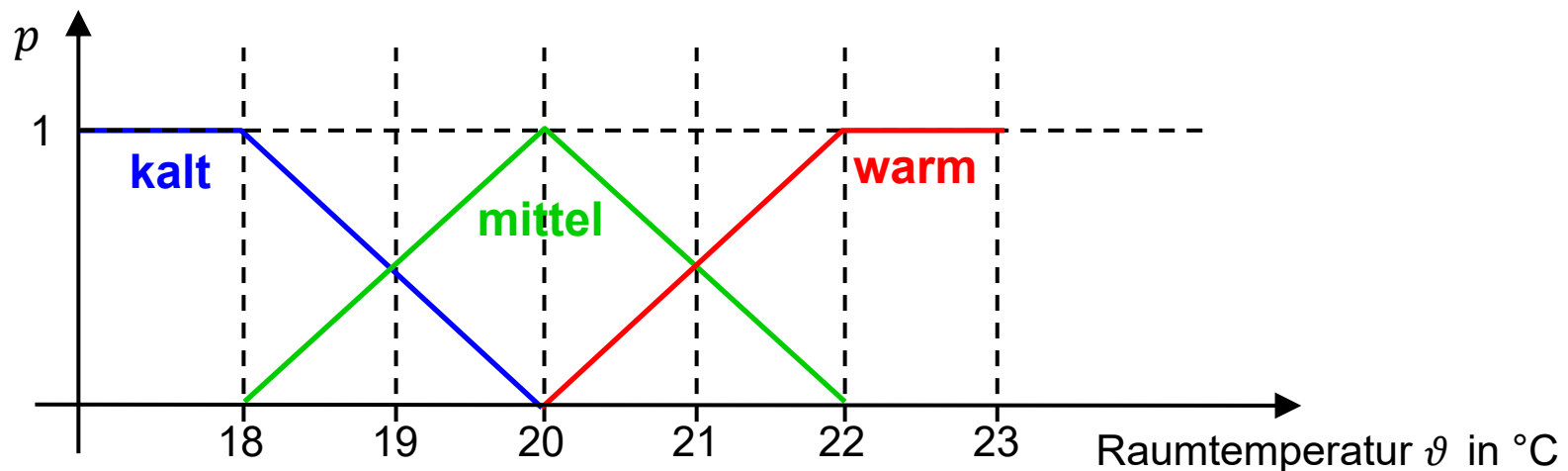
2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Beschreibung von Fuzzy-Mengen:

Die Zugehörigkeit eines Elements zu einer Fuzzy-Menge, wie auch die Gültigkeit der dazugehörigen Aussagelogik wird durch einen Wert von 0 bis 1 beschrieben:

- 0 → Element gehört nicht zur Menge / Die Aussage „Element gehört zur Menge“ ist falsch.
- 1 → Element gehört zur Menge / Die Aussage „Element gehört zur Menge“ ist wahr.
- p → Element gehört zu $p * 100\%$ zur Menge /
Die Aussage „Element gehört zur Menge“ ist zu $p * 100\%$ wahr.

Bei numerische Größen kann p über eine Zugehörigkeitsfunktion beschrieben werden:



2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Anmerkung 1:

Der klassische Mengenbegriff kann als Spezialfall des neuen, unscharfen Mengenbegriffs betrachtet werden. Wenn die Zugehörigkeitsfunktion einer Menge nur die Werte 0 und 1 annimmt, so ist diese Menge wieder scharf abgegrenzt.

Anmerkung 2:

Es gibt Parallelen zwischen der Fuzzy-Logik und der Wahrscheinlichkeitsrechnung, da der Zugehörigkeitswert p auch Wahrscheinlichkeit einer Zugehörigkeit interpretiert werden kann.

Anmerkung 3:

Es gibt auch Parallelen zu den RBF-Netzen, wenn man die Zugehörigkeitsfunktionen als RBF-Funktionen auffasst → Kombination der Techniken: Neuro-Fuzzy-Systeme.

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Fuzzy-Logik (unscharfe Logik):

- Um mit den „linguistischen“ Termen arbeiten zu können sind meist Verknüpfungen notwendig, die bei klassischer Aussagenlogik über die „üblichen“ logischen Verknüpfungen NICHT, UND und ODER erfolgt.
- Unscharfe Mengen und die resultierenden unscharfen Aussagen mit Wahrheitswerten $p = 0 \dots 1$ bedingen also eine Erweiterung der logischen Verknüpfungen.
- Die Definition dieser für unscharfen Aussagen erweiterten logischen Verknüpfungen ist nicht eindeutig, d. h. es gibt mehrere gebräuchliche Definition.
- Meist werden die erweiterten logische Verknüpfungen so gewählt, dass sie für den Spezialfall der klassischen Mengen und Aussagenlogik ($p = 0$ oder $p = 1$) mit den klassischen Logikoperationen übereinstimmen.

Def.: **NICHT** (Invertierung):

Sei p der Wahrheitswert der unscharfen Aussage " $A \in M$ " (Zugehörigkeitswert von A auf M) dann sei $\bar{p}$ als Wahrheitswert der unscharfen Aussage " $A \notin M$ "

$$\bar{p} = 1 - p$$

Beispiele: $p = 0$ (" $A \in M$ " ist falsch) $\rightarrow \bar{p} = 1$ (" $A \notin M$ " ist wahr)
 $p = 0,5$ (" $A \in M$ " ist 50% wahr) $\rightarrow \bar{p} = 0,5$ (" $A \notin M$ " ist 50% wahr)

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Def.: **UND**-Verknüpfung:

Seien p_A und p_B die Wahrheitswerte der unscharfen Aussagen " $A \in M$ " **und** " $B \in M$ "
dann sei $p_{A\&B}$ als Wahrheitswert der unscharfen Aussage " $A \in M$ UND $B \in M$ "

$$p_{A\&B} = \min(p_A, p_B)$$

$$\begin{array}{ll} \text{Beispiele: } p_A = 1 \text{ und } p_B = 0 & \rightarrow p_{A\&B} = 0 \\ p_A = 1 \text{ und } p_B = 0,5 & \rightarrow p_{A\&B} = 0,5 \end{array}$$

Alternativ:

$$p_{A\&B} = p_A \cdot p_B$$

Def.: **ODER**-Verknüpfung:

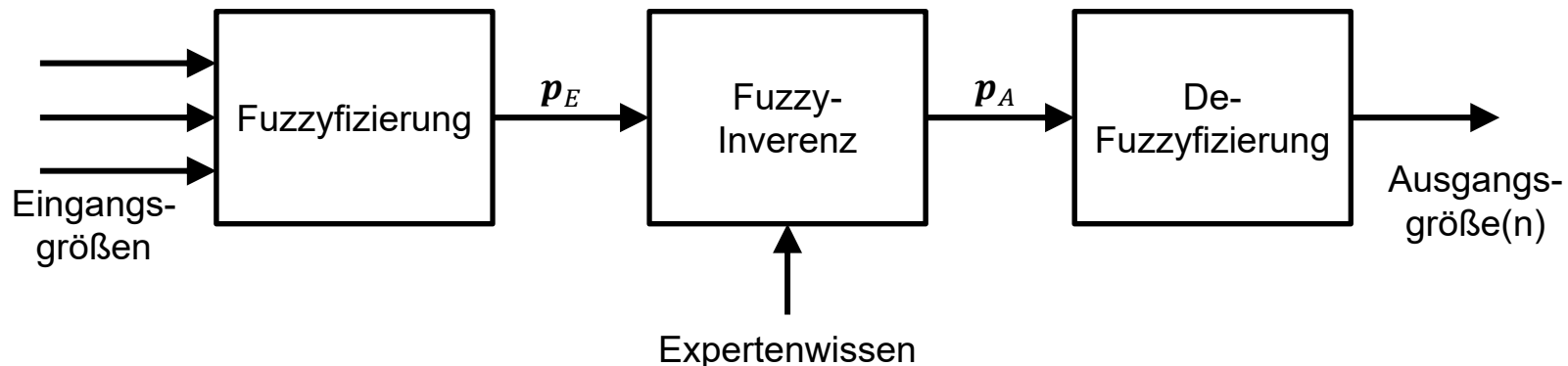
Seien p_A und p_B die Wahrheitswerte der unscharfen Aussagen " $A \in M$ " **oder** " $B \in M$ "
dann sei $p_{A||B}$ als Wahrheitswert der unscharfen Aussage " $A \in M$ ODER $B \in M$ "

$$p_{A||B} = \max(p_A, p_B)$$

$$\begin{array}{ll} \text{Beispiele: } p_A = 1 \text{ und } p_B = 0 & \rightarrow p_{A||B} = 1 \\ p_A = 1 \text{ und } p_B = 0,5 & \rightarrow p_{A||B} = 1 \end{array}$$

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Aufbau eines Fuzzy-Systems (Fuzzy-Reglers):



Fuzzyfizierung:

Ermittlung der Zugehörigkeitswerte p_E (hier Vektor aller Zugehörigkeitswerte) zu den die linguistische Terme beschreibenden unscharfen Mengen. Dies geschieht je nach dem für mehrere Eingangsgrößen, die allerdings unabhängig mit jeweils passenden linguistischen Termen und unscharfen Mengen behandelt werden.

Fuzzy-Inferenz (Inferenz → Schlußfolgerung)

Verknüpfung der aus den einzelnen Eingangsgrößen resultierenden unscharfen Aussagen durch „Wenn-Dann-Regeln“. Diese ergibt unter Verwendung der für unscharfe Logik erweiterten logischen Verknüpfungen die Zugehörigkeitswerte p_A der Ausgangsgröße. Der Idee nach enthält das in der Fuzzy-Inferenz verankerte „Regelwerk“ das umgesetzte Fachwissen des Experten.

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

De-Fuzzyfizierung

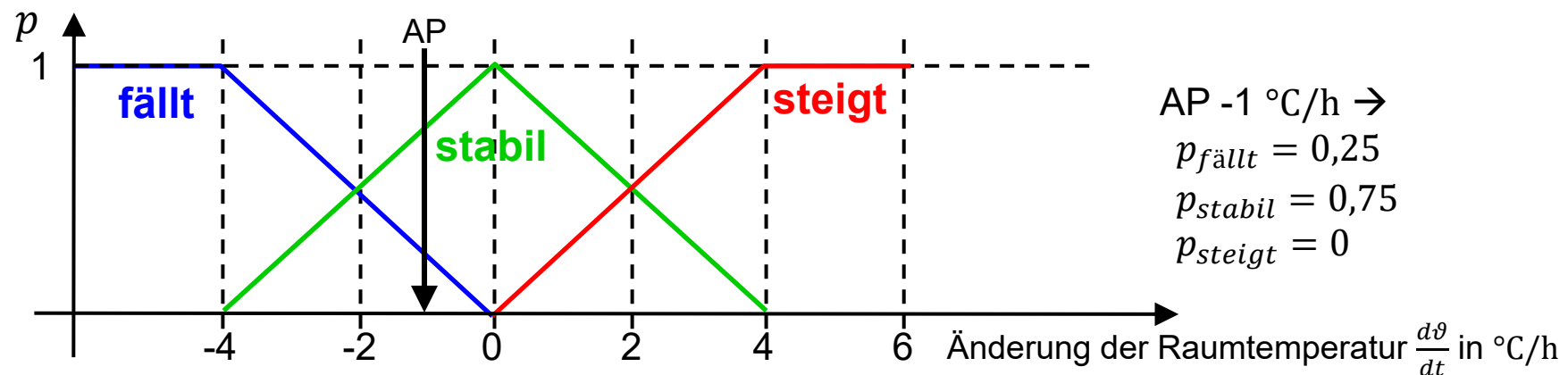
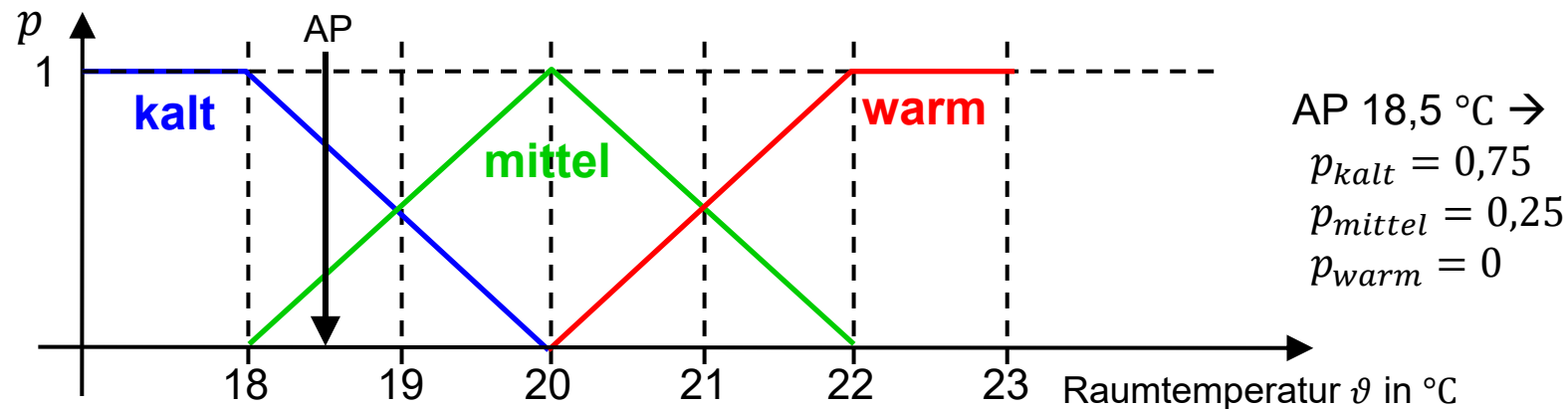
Gewinnung einer numerischen Ausgangsgröße aus den Zugehörigkeitswerte p_A zu der die linguistische Terme beschreibenden unscharfen Mengen. Diese Umkehrung der Fuzzyfizierung kann nicht über einen formal rein inversen Prozess erfolgen, da aus der Fuzzy-Inverenz i. d. R. eine Kombination von Zugehörigkeitswerte p_A für die einzelnen Fuzzy-Mengen resultiert, die keinem Wert der Ausgangsgröße zuzuordnen ist. Ein übliches Verfahren ist daher z. B. die Bildung des Flächenschwerpunkts.

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Beispiel: **Regelung der Raumtemperatur**

Eingangsgrößen: Raumtemperatur ϑ in °C und Änderung der Raumtemperatur $\frac{d\vartheta}{dt}$ in °C/h

Ausgangsgröße: Heizleistung P in kW



2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Fuzzy-Inferenz: Beispielregelwerk:

Wenn Raumtemperatur „kalt“ und „fällt“	dann „stark“ heizen	oder
wenn Raumtemperatur „kalt“ und „stabil“	dann „mittel stark“ heizen	oder
wenn Raumtemperatur „mittel“ und „fällt“	dann „mittel stark“ heizen	oder
wenn Raumtemperatur „kalt“ und „steigt“	dann „mittel“ heizen	oder
wenn Raumtemperatur „mittel“ und „stabil“	dann „mittel“ heizen	oder
wenn Raumtemperatur „warm“ und „fällt“	dann „mittel“ heizen	oder
wenn Raumtemperatur „mittel“ und „steigt“	dann „schwach“ heizen	oder
wenn Raumtemperatur „warm“ und „stabil“	dann „schwach“ heizen	oder
wenn Raumtemperatur „warm“ und „steigt“	dann „nicht“ heizen.	

Die Regeln lassen sich tabellarisch übersichtlicher darstellen:

	kalt	mittel	warm
fällt	stark	mittelstark	mittel
stabil	mittelstark	mittel	schwach
steigt	mittel	schwach	nicht

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Hier im AP:

Wenn Raumtemperatur „kalt“ und „fällt“	dann „stark“ heizen	oder
wenn Raumtemperatur „kalt“ und „stabil“	dann „mittel stark“ heizen	oder
wenn Raumtemperatur „mittel“ und „fällt“	dann „mittel stark“ heizen	oder
...		
wenn Raumtemperatur „mittel“ und „stabil“	dann „mittel“ heizen	oder
...		

Die Ver**ODER**erung muss nur bei identischen linguistischen Termin für den Ausgang explizit berücksichtigt werden (hier bei „mittel stark“):

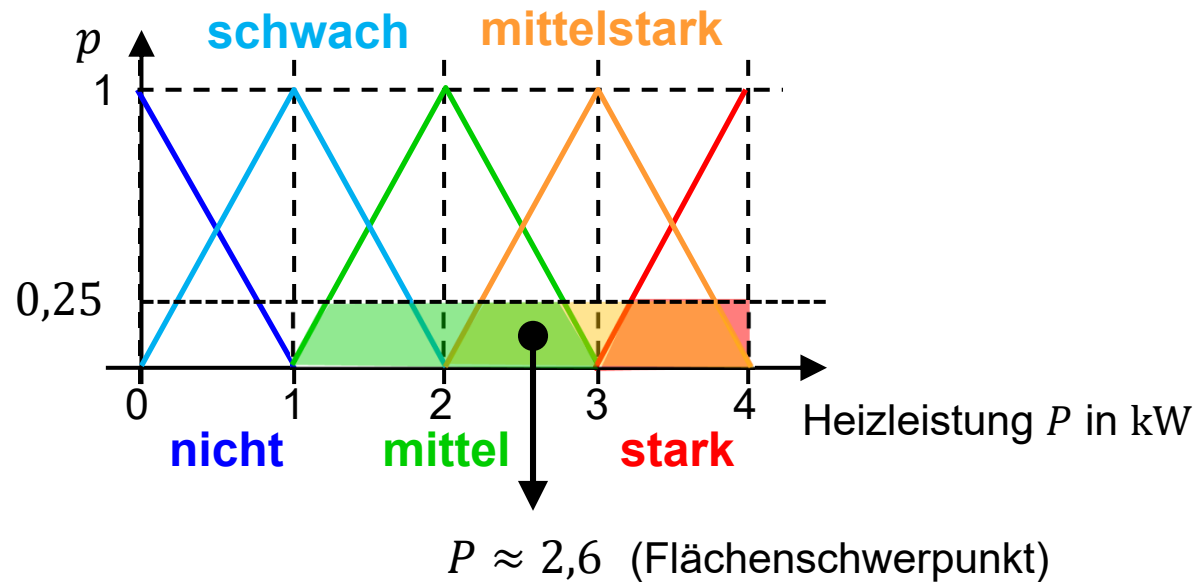
- „kalt“ **und** „fällt“ dann „stark“ $\rightarrow p_{stark} = \min(p_{kalt}, p_{fällt}) = 0,25$
- „kalt“ **und** „stabil“ dann „mittel stark“ **oder** „mittel“ **und** „fällt“ dann „mittel stark“
 $\rightarrow$ „kalt“ **und** „stabil“ **oder** „mittel“ **und** „fällt“ dann „mittel stark“
 $\rightarrow p_{mittelstark} = \max(\min(p_{kalt}, p_{stabil}), \min(p_{mittel}, p_{fällt})) = \max(0,25, 0,25) = 0,25$
- „mittel“ **und** „stabil“ dann „mittel“ $\rightarrow p_{mittel} = \min(p_{mittel}, p_{stabil}) = 0,25$

2. Approximation von Funktionen – 2.5 Fuzzy-Logik

De-Fuzzifizierung:

Werte: $p_{stark} = 0,25$, $p_{mittelstark} = 0,25$, $p_{mittel} = 0,25$

→ Kein direkt korrespondierender Wert für die Heizleistung



2. Approximation von Funktionen – 2.5 Fuzzy-Logik

Anmerkungen:

- Es können auch gewichtete Regeln verwendet werden um auszudrücken, dass einige Regeln weniger wichtig sind als andere.
- Die Regelwerke müssen nicht immer alle kombinatorisch möglichen Verknüpfungen für die linguistischen Terme beinhalten.